

第15-16课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

第 15-16课 PDF 解析整理版

第 15 课+第16课：工程化升级 语义匹配+规则约束

[要点] 两课核心关系

第15课与第16课构成了Schema Linking从"表级召回"到"表+字段级混合匹配"的关键细化：

- 第15课（工程化升级）：把第14课手写的"内存dict + 暴力循环"换成LangChain + ChromaDB，用三层抽象（Embeddings / VectorStore / Retriever）建立后续所有课复用的向量检索基础设施；并通过故意加入 4 张不相关表（HR考勤 / IoT告警 / 法务合同 / 仓储温度）证明：在不相关表描述中加入"不用于收入分析""不参与销售统计"等排除性表述，可以被Embedding 向量化、自然被推到检索末位。
- 第16课（语义匹配+规则约束）：在15课表召回的基础上，把"选对表"进一步细化到"选对字段"。在构造字段描述时引入"业务含义 + 使用场景 + 区分说明 + 枚举值"四个维度，并首次引入"业务规则"作为混合评分的第二维度（白名单/黑名单/条件触发）——这是14-15 课"纯向量"范式里完全没有的新概念。

[链接] 逻辑衔接点

- 基础设施复用：第16课的向量化方案与第15课完全相同，只换了一个collection（table_descriptions → field_descriptions）和持久化目录（chroma_db/tables/ → chroma_db/fields/），相同的 get_embeddings() + get_vectorstore() + _cosine_relevance_score_fn 模式——这正是第15课"三层抽象"带来的可替换性收益。
- 候选范围限定：第16课在字段匹配时不是对全量 35 个字段做全局搜索，而是把第15课表召回返回的 Top-K 表名作为 \$in 过滤条件——把"选对表"和"选对字段"两件事解耦，前者负责缩小候选集，后者负责在候选集内精排。
- 混合评分的接口预留：第16课的 final_score = (1 - α) × embedding_score + α × rule_score 公式和 BUSINESS_RULES 列表结构，给第18课 Schema Linking Pipeline 留了一个"语义+规则"的统一评分契约；rule_weight（默认 0.3）将成为整个 pipeline 的关键超参数。

- 标识约定：第16课引入 `field_key` = "表名. 字段名"（如 `sales_orders.net_amount`）作为字段的唯一标识，这个格式是第18课 Pipeline 把"表召回结果"和"字段匹配结果"做关联的连接键。

两节课的关系 / 总结

维度

第 15 课（LangChain + ChromaDB 工程化）

第 16 课（字段语义匹配）

所处阶段

第四阶段《让系统"理解业务"》第三节

第四阶段《让系统"理解业务"》第四节

核心问题

手写向量检索有四个工程缺陷（无持久化 / 暴力 $O(n)$ / 无元数据过滤 / 无批量）

表召回选对了表，表内几十个字段依然会选错（`gross_amount` vs `net_amount`）

核心概念

HNSW / 三层抽象（Embeddings / VectorStore / Retriever）

字段描述四维度 / 业务规则（whitelist / blacklist / conditional）

关键产出

`get_embeddings()` + `get_vectorstore()` + ChromaDB 持久化目录

`build_field_index()` + BUSINESS_RULES + `retrieve_schema()` 串联接口

在链路中的位置

Schema Linking 的"基础设施层"

Schema Linking 的"字段匹配子模块"

下一课接力

列出后续 16 / 19 / 20 课的 collection 复用方式

把"Join 路径推理"作为下一步接力点

一句话关系总结：

- 第 15 课让系统"把表选对后能落盘、能工程化"（LangChain 三层抽象 + ChromaDB 持久化 + 故意加入 4 张不相关表验证过滤能力）；
- 第 16 课让系统"在选对的表里把字段也选对"（丰富描述 + 候选范围限定 + 业务规则混合评分，把 gross_amount vs net_amount 这类歧义从"模型靠运气"变成"规则强制 + 语义引导"双保险）；
- 衔接关系：第 16 课完全复用第 15 课的 Embeddings/VectorStore 模式，只换 collection；并把 field_key 命名约定和"语义+规则"评分公式作为第 18 课 Schema Linking Pipeline 的两个关键契约。

第 15 课：从手写到框架——LangChain + ChromaDB 向量检索工程化

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——Schema Linking + RAG》第三节
- 本课核心问题：第 14 课用 80 行代码实现了向量检索的完整闭环，验证了"语义匹配"的可行性。但该方案有四个工程缺陷：无持久化、暴力搜索、无元数据过滤、无批量操作。本课引入 LangChain + ChromaDB，用工程化方案解决这些问题，并建立后续所有课程复用的向量检索基础设施。

本节课目标：

- 理解为什么需要向量数据库（对比手写方案的四个缺陷）
- 理解 LangChain 的三层抽象：Embeddings → VectorStore → Retriever
- 能用 LangChain + ChromaDB 重构表级召回模块，实现持久化 + 元数据过滤
- 建立后续第 16-20 课直接复用的技术底座

开场引入

回顾第 14 课的手写方案：

```
# 向量存储：一个 Python dict
index = {"dim_customers": [0.12, -0.34, ...], "sales_orders": [...], ...}

# 检索：for 循环 + 余弦计算
for table_name, embedding in index.items():
    score = cosine_similarity(query_vec, embedding)
```

这段代码能工作，但有一个致命问题——进程退出后，所有向量都丢了。下次启动需要重新调 Embedding API 生成所有向量（5 张表要调 5 次 API，50 张表就是 50 次）。

更根本的问题是：如果要支持更多维度的检索（比如“只在事实表范围内搜索”“按主题域过滤”），用 Python dict 无法实现高效的条件过滤。

一、为什么需要向量数据库

第 14 课方案的四个缺陷对应向量数据库的四个核心能力：

手写方案缺陷

向量数据库解决方案

内存 dict，重启丢失

持久化存储：向量写入磁盘，重启后直接加载

for 循环暴力搜索 $O(n)$

ANN 近似最近邻：HNSW 等算法， $O(\log n)$

无法按条件预筛选

元数据过滤：先按 metadata 缩小候选集，再做向量搜索

单条调 API

批量写入：一次性将多条文档向量化并写入

向量数据库选型

数据库

部署方式

适合规模

本课程选择理由

ChromaDB

嵌入式（无需启动服务）

万级以内

零配置、Python 原生、新生友好

FAISS

嵌入式

百万级

高性能但 API 偏底层，不支持元数据过滤

Milvus

分布式服务

亿级

生产级但部署复杂

Qdrant

独立服务

千万级

功能强大但需额外运维

[要点] 本课程选择 ChromaDB: 嵌入式部署 (import 即用, 无需启动独立服务)、支持持久化到本地目录、支持元数据过滤、Python API 极其简洁。对于当前 5-50 张表的检索规模, ChromaDB 绰绰有余。

HNSW 算法简介

ChromaDB 内部使用 HNSW (Hierarchical Navigable Small World) 算法做近似最近邻搜索。

直觉理解: 想象一个多层的"跳表"。顶层只有少量节点(跨大步快速定位区域), 底层包含所有节点(精确搜索)。检索时从顶层开始、逐层下降、每层在邻居中找最近的节点, 最终定位到底层的精确结果。

好处: 复杂度从 $O(n)$ 降到 $O(\log n)$, 在百万级向量中仍能保持毫秒级响应。

对于本课程的规模 (5-50 条向量), 暴力搜索和 HNSW 的性能差异可以忽略。但理解 HNSW 的存在, 是为了说明"向量数据库不只是一个持久化的 dict, 它内部有专门的索引结构"。

二、LangChain 的三层抽象

为什么用 LangChain

直接使用 ChromaDB 原生 API 完全可以实现功能。但 LangChain 提供了一层统一抽象，解决的是"可替换性"问题：

如果未来需要：

- 把 Embedding 模型从 OpenAI 换成本地 BGE → 只改 Embeddings 实例
- 把向量数据库从 ChromaDB 换成 FAISS → 只改 VectorStore 实例
- 把检索策略从单次查询换成多查询融合 → 只改 Retriever 实例

调用方代码完全不变，这就是抽象层的价值。

三层分别做什么

层

职责

本课使用

Embeddings

将文本转为向量

OpenAIEmbeddings （OpenAI 兼容接口）

VectorStore

存储向量 + 执行搜索

Chroma （ChromaDB）

Retriever

封装检索策略（Top-K、过滤、重排序）

直接调用 VectorStore 的 similarity_search_with_relevance_scores

三、环境准备

安装依赖

```
uv add langchain-openai langchain-chroma chromadb
```

安装完成后 `pyproject.toml` 中应新增：

```
dependencies = [
    # ... 已有依赖...
    "langchain-openai",
    "langchain-chroma",
    "chromadb",
]
```

验证安装

```
uv run python -c "from langchain_chroma import Chroma; print('ChromaDB OK')"
```

```
uv run python -c "from langchain_openai import OpenAIEmbeddings; print('OpenAIEmbeddings OK')"
```

两条命令均无报错即安装成功。

四、用 LangChain + ChromaDB 重构表级召回

1. 整体架构

2. 构建 Embeddings 实例

```
from langchain_openai import OpenAIEmbeddings
from config import LLM_CONFIG

embeddings = OpenAIEmbeddings(
    model=LLM_CONFIG["embedding_model"],
    base_url=LLM_CONFIG["base_url"],
    api_key=LLM_CONFIG["api_key"],
)
```

使用项目已有的 `.env` 配置（`OPENAI_API_KEY`、`OPENAI_BASE_URL`、`EMBEDDING_MODEL`），无需额外配置。

3. 创建 VectorStore

```
from langchain_chroma import Chroma

def _cosine_relevance_score_fn(distance: float) -> float:
    """将 ChromaDB 的 cosine 距离转换为余弦相似度"""
    return 1 - distance

vectorstore = Chroma(
    collection_name="table_descriptions",
    embedding_function=embeddings,
    persist_directory="./chroma_db/tables",
    collection_metadata={"hnsw:space": "cosine"}, # 使用 cosine 距离
    relevance_score_fn=_cosine_relevance_score_fn, # 转换为余弦相似度
)
```

三个关键配置项：

- collection_metadata={"hnsw:space": "cosine"}：指定 ChromaDB 使用 cosine 距离而非默认的 L2 距离。如果不指定，ChromaDB 默认使用 L2 距离
- relevance_score_fn：将 ChromaDB 返回的 cosine 距离（1 - 余弦相似度）转换回余弦相似度（1 - distance），使分数与第 14 课手写版一致（值域-1到1）
- persist_directory：持久化到本地磁盘的目录，重启后数据不丢

4. 构建 Document 并写入索引

虽然我们在本业务场景下精选了 5
张业务相关的表，但真实企业数据库通常有几十上百张表，其中大量表与业务分析无关。本课在 5
张相关表之外，额外加入 4 张不相关表来验证检索的区分能力：

表名	
类型	
与业务分析的关系	
dim_customers	
维度表	
[OK] 相关	
dim_products	
维度表	
[OK] 相关	
sales_orders	
事实表	
[OK] 相关	
exchange_rates	
参考表	
[OK] 相关	
finance_expenses	

事实表	
[OK] 相关	
hr_attendance_records	
HR系统	
[X] 不相关	
iot_device_alerts	
设备运维	
[X] 不相关	
legal_contract_archive	
法务系统	
[X] 不相关	
warehouse_temperature_logs	
仓储管理	
[X] 不相关	

不相关表的描述中刻意加入了"不用于收入分析""不参与销售统计"等排除性表述——这些表述会被 Embedding 向量化，使得不相关表在检索时天然排名靠后。

LangChain 用 Document 对象承载检索内容：

```
from langchain_core.documents import Document

doc = Document(
    page_content="客户维度表：存储客户基本信息...", # 被向量化和检索的文本
    metadata={
        "table_name": "dim_customers", # 元数据：不被向量化，但可用于过滤
        "domain": "维度表",
        "key_fields": "customer_id, customer_name, ...",
    },
)
```

[要点] Document 有两个核心字段：page_content（被向量化的文本，决定检索效果）和 metadata（不被向量化的附加信息，用于过滤和返回）。这两者的区别——放在 page_content 里的内容影响语义匹配，放在 metadata 里的不影响匹配但可以辅助筛选。

批量写入：

```
documents = []
for table_name, meta in TABLE_METADATA.items():
    doc = Document(
        page_content=meta["description"],
        metadata={
            "table_name": table_name,
            "domain": meta["domain"],
            "key_fields": meta["key_fields"],
        },
    )
    documents.append(doc)

# 一次性写入，ids 用表名作为唯一标识（避免重复插入）
vectorstore.add_documents(documents, ids=list(TABLE_METADATA.keys()))
```

5. 执行检索

```
# 基础检索：返回最相关的 Top-3
results = vectorstore.similarity_search_with_relevance_scores(
    "欧洲市场最近三个月的销售额", k=3
)
for doc, score in results:
    print(f"{doc.metadata['table_name']} → 相似度: {score:.4f}")
```

6. 元数据过滤

ChromaDB 支持在向量搜索前先按 metadata 条件过滤候选集：

```
# 仅在"事实表"范围内检索
results = vectorstore.similarity_search_with_relevance_scores(
    "上个月的收入", k=3, filter={"domain": "事实表"}
)
```

这在企业场景中非常有用——比如已经确定问题涉及"费用分析"，可以预先排除维度表，减少误召回。

五、对比：手写版 vs LangChain 版

维度

手写版（第 14 课）

LangChain + ChromaDB（本课）

持久化

[X] 内存 dict，重启丢失

[OK] 本地目录，永久保存

搜索性能

$O(n)$ 暴力遍历

$O(\log n)$ HNSW

元数据过滤

[X] 不支持

[OK] 按 domain 等字段过滤

批量操作

[X] 逐条 API

[OK] add_documents 批量写入

模型可替换

[X] 绑定 OpenAI

[OK] 换 Embeddings 实例即可

数据库可替换

[X] 绑定 dict

[OK] 换 VectorStore 即可

代码量

~80 行

~60 行（更少，因为框架封装了细节）

运行演示

完整代码文件路径：code/chatbi_mvp/table_retriever.py

```
uv run table_retriever.py
```

首次运行输出：

```
=====
LangChain + ChromaDB 表级召回演示
=====
```

```
索引构建完成：9 张表已写入 ChromaDB
```

```
持久化路径：./chroma_db/tables
```

```
问题：欧洲市场最近三个月的销售额是多少？
```

召回结果：

• dim_customers	相似度：0.2834	[维度表]
• sales_orders	相似度：0.2739	[事实表]
• finance_expenses	相似度：0.2417	[事实表]
• dim_products	相似度：0.2348	[维度表]
• exchange_rates	相似度：0.2083	[参考表]

问题：各产品线的毛利率

召回结果：

[OK] dim_products	相似度：0.4996	[维度表]
• finance_expenses	相似度：0.3806	[事实表]
• dim_customers	相似度：0.3201	[维度表]
• sales_orders	相似度：0.2784	[事实表]
• warehouse_temperature_logs	相似度：0.2471	[仓储管理]

问题：上个月研发费用和销售费用对比

召回结果：

[OK] finance_expenses	相似度：0.5278	[事实表]
• sales_orders	相似度：0.3160	[事实表]
• dim_products	相似度：0.3124	[维度表]
• dim_customers	相似度：0.2807	[维度表]
• exchange_rates	相似度：0.2713	[参考表]

问题：查询已完成订单的总数量

召回结果：

[OK] sales_orders	相似度：0.4858	[事实表]
• dim_customers	相似度：0.2882	[维度表]
• exchange_rates	相似度：0.2579	[参考表]
• warehouse_temperature_logs	相似度：0.2378	[仓储管理]
• legal_contract_archive	相似度：0.2257	[法务系统]

问题：按客户类型统计收入，需要换算成人民币

召回结果：

[OK] exchange_rates	相似度：0.5151	[参考表]
[OK] dim_customers	相似度：0.4373	[维度表]
• sales_orders	相似度：0.3579	[事实表]
• finance_expenses	相似度：0.3335	[事实表]
• hr_attendance_records	相似度：0.3296	[HR系统]

效果分析

从输出中可以观察到两个关键现象：

相关表始终排名靠前——

- "各产品线的毛利率"： dim_products 排第一（0.4996），远高于不相关的仓储日志表（0.2471）
- "研发费用和销售费用对比"： finance_expenses 排第一（0.5278），4 张不相关表全部排在后面
- "按客户类型统计收入，换算人民币"： exchange_rates 和 dim_customers 排前两位——"换算成人民币"触发了汇率表的语义匹配

不相关表被有效压低——

每个查询中，`hr_attendance_records`、`iot_device_alerts`、`legal_contract_archive`、`warehouse_temperature_logs` 这 4 张不相关表的相似度始终处于末位（0.12至0.25 区间），与相关表（0.28至0.53 区间）有明显分界。

[要点] 关键发现：不相关表描述中“不用于收入分析”“不参与销售统计”等排除性表述，确实帮助 Embedding 将它们推到了低分区域。这说明表描述文本的质量不仅影响“能不能被召回”，还影响“能不能被正确排除”。

再次运行（进程重启后），输出：

```
已有索引数据（9 条），跳过重建。如需重建请传入 force_rebuild=True
```

不再重新调用 Embedding API——这就是持久化的价值。

六、相似度阈值与 Top-K 策略

阈值设定

向量检索返回的分数不是绝对的“正确率”，而是“相对距离”。设定阈值需要在两个风险之间取舍：

- 阈值太高（如 > 0.4 ）：只保留非常相关的结果 → 可能漏掉实际需要的表
- 阈值太低（如 > 0.1 ）：几乎所有表都能通过 → 噪声太多，等于没过滤

经验值：对于表级召回场景，阈值设在 0.2至0.3 之间通常是合理的起点。具体值需要通过实际测试用例调优——不同 Embedding 模型的分数分布可能差异很大（本课程使用的模型，相关表分数集中在 0.28至0.53 区间，不相关表在 0.12至0.25 区间，因此阈值设为 0.2 较为合理）。

Top-K 选择

对于当前 9 张表（5 相关 + 4 不相关）的场景，`top_k=5` 足够覆盖所有可能相关的表，同时也能观察到不相关表的分数分布。

如果企业扩展到 50 张表，建议 `top_k=5~8`，搭配阈值过滤，保证既有足够的召回覆盖，又不引入过多噪声。

七、后续课程如何复用

本课建立的技术底座—— `get_embeddings()` + `get_vectorstore()` + ChromaDB 持久化目录——将在后续课程中被直接复用：

后续课程

复用方式

第 16 课：字段匹配

新建 chroma_db/fields collection，存字段描述向量

第 19 课：指标 RAG

新建 chroma_db/indicators collection，存指标定义向量

第 20 课：全链路融合

并行查询多个 collection，合并结果

核心模式完全相同：构建 Document → 写入 collection → 检索时指定 collection 名称。

小结

本节课完成了向量检索的"工程化升级"：

下一课将在此基础上实现字段级语义匹配——从"选对表"进一步细化到"选对字段"。

第 16 课：字段语义匹配——解决"amount 到底是哪种金额"

课程定位与目标

课程定位：

- 所处阶段：第四阶段《让系统"理解业务"——Schema Linking + RAG》第四节
- 本课核心问题：第 15 课完成了表级召回——能选对表，但表内有几十个字段，模型依然可能选错。典型歧义：gross_amount vs net_amount（含税 vs 不含税）、region vs country（大区 vs 国家）。本课在表召回基础上，实现字段级的语义匹配，并引入业务规则做"混合评分"。

本节课目标：

- 理解为什么表召回之后还需要字段匹配
- 掌握字段描述文本的构造策略（决定 Embedding 质量的关键）
- 实现"向量相似度 + 业务规则"的混合匹配方案
- 将表召回 + 字段匹配串联，输出精简 Schema 片段

开场引入

回顾第 15 课的成果：系统能准确召回与问题相关的表。例如，“欧洲市场最近三个月的销售额”→ 召回 sales_orders + dim_customers。

但召回表之后，模型面对 sales_orders 的 13 个字段，会发生什么？

问题：“查询上个月的销售额”

sales_orders 中有两个金额字段：

- gross_amount（含税总额）
- net_amount（不含税收入）

模型选了 gross_amount —— 错！

业务规定“销售额”= 不含税收入 = net_amount

再看一个：

问题：“欧洲市场的客户收入”

dim_customers 中有两个地理字段：

- country（具体国家：Germany, France...）
- region（大区：欧洲、北美...）

模型选了 country —— 错！

“欧洲市场”对应的是 region = '欧洲'，不是某个具体国家

这类错误的根源：字段名和字段注释不足以消除歧义，需要更丰富的语义描述和业务规则约束。

一、字段描述文本构造策略

为什么字段描述很重要

向量检索的本质是“把用户问题和候选项都变成向量，然后比较距离”。候选项的文本质量直接决定匹配质量——如果字段描述太简短（如只有字段名 net_amount），Embedding 无法区分它和 gross_amount 的业务差异。

构造原则

好的字段描述应包含四个维度：

维度

作用

示例

字段名 + 类型

基本标识

net_amount, DECIMAL (12, 2)

业务含义

区分同类字段的核心语义

"不含税收入（财务口径的销售额）"

使用场景

告诉模型什么时候该用这个字段

"日常说的'收入''销售额'统一使用此字段"

区分说明

消除与易混淆字段的歧义

"注意：除非明确要求含税，否则不用 gross_amount"

对比：简陋描述 vs 丰富描述

简陋版（第 5 课 Schema 中的写法）：

- net_amount DECIMAL (12, 2) 不含税收入（财务口径的销售额）
- gross_amount DECIMAL (12, 2) 含税总额

[图：对比：简陋描述 vs 丰富描述——同一字段在不同详细度下的 Embedding 分布示意]

Embedding 很难从这两句话中区分"销售额"应该映射到哪个字段。

丰富版（本课方案）：

```
"sales_orders.net_amount": {
  "description": (
    "不含税收入（财务口径的销售额），DECIMAL (12, 2)。"
    "这是业务中'收入''销售额''营业收入'的标准字段。"
    "毛利计算：毛利 = net_amount - (material_cost + labor_cost) * quantity。"
    "统计收入时必须同时过滤 order_status = 'completed'。"
  ),
}

"sales_orders.gross_amount": {
  "description": (
    "含税总额，DECIMAL (12, 2)。包含增值税的订单总金额。"
    "注意：除非用户明确要求'含税金额'，否则不应使用此字段。"
    "日常说的'收入''销售额'统一使用 net_amount（不含税收入）。"
  ),
}
```

丰富版的描述中，"收入""销售额"等关键词出现在 net_amount 的描述里，且 gross_amount

的描述中明确标注了"不应使用"的条件——这使得 Embedding 向量能准确反映业务语义。

构造模板

```
{字段名}, {数据类型}。  
{业务含义的一句话解释}。  
{使用场景/计算公式}。  
{与易混淆字段的区分说明}。
```

对于有枚举值的字段（如 customer_type、order_status），还要加上：

```
枚举值: {value1} / {value2} / {value3}。
```

枚举值进入 Embedding 后，能帮助系统理解"OEM 客户"→ customer_type、"已完成订单"→ order_status = 'completed' 这类映射。

二、字段向量化与 ChromaDB 存储

复用第 15 课基础设施

本课的向量化方案与第 15 课完全相同，只是换了一个 collection：

维度

第 15 课（表召回）

本课（字段匹配）

Collection 名

table_descriptions

field_descriptions

持久化目录

chroma_db/tables/

chroma_db/fields/

Document 内容

表的业务描述

字段的业务描述

Metadata

table_name, domain

table_name, field_name, field_key

数据量

5 条 (5 张表)

~35 条 (5 表 × 平均 7 字段)

```
CHROMA_PERSIST_DIR = os.path.join(os.path.dirname(__file__), "chroma_db", "fields")

def _cosine_relevance_score_fn(distance: float) -> float:
    """将 ChromaDB 的 cosine 距离转换为余弦相似度 (与第 15 课一致)"""
    return 1 - distance

def get_vectorstore() -> Chroma:
    return Chroma(
        collection_name="field_descriptions",
        embedding_function=get_embeddings(),
        persist_directory=CHROMA_PERSIST_DIR,
        collection_metadata={"hnsw:space": "cosine"},
        relevance_score_fn=_cosine_relevance_score_fn,
    )
```

索引构建

为所有表的所有字段创建 Document 并写入：

```
def build_field_index(force_rebuild: bool = False) -> Chroma:
    vectorstore = get_vectorstore()
    existing = vectorstore._collection.count()

    if existing > 0 and not force_rebuild:
        print(f"字段索引已存在 ({existing} 条)，跳过重建。")
        return vectorstore

    if force_rebuild and existing > 0:
        existing_ids = vectorstore._collection.get()["ids"]
        if existing_ids:
            vectorstore._collection.delete(ids=existing_ids)
            print("已清空旧索引数据")

    documents = []
    ids = []
    for field_key, meta in FIELD_METADATA.items():
        doc = Document(
            page_content=meta["description"],
            metadata={
                "table_name": meta["table"],
                "field_name": meta["field"],
                "field_key": field_key, # 如 "sales_orders.net_amount"
                "domain": meta["domain"],
            },
        )
```

```

    )
    documents.append(doc)
    ids.append(field_key)

vectorstore.add_documents(documents, ids=ids)
print(f"字段索引构建完成: {len(documents)} 个字段已写入 ChromaDB")
return vectorstore

```

[要点] 注意 `field_key` 的命名约定：表名. 字段名（如 `sales_orders.net_amount`）。这个格式在后续的 Schema Linking Pipeline 中作为字段的唯一标识，用于连接表召回和字段匹配的结果。

候选范围限定

字段匹配不是在全部 35 个字段中搜索，而是只在表召回结果对应的字段范围内搜索。

```

# 先表召回
tables = retrieve_tables(query) # → ["sales_orders", "dim_customers"]

# 再在候选表范围内做字段匹配
search_kwargs = {"k": 10}
if candidate_tables:
    search_kwargs["filter"] = {"table_name": {"$in": candidate_tables}}

```

为什么不在全部字段中搜索？

1. 减少噪声：避免无关表的字段干扰（如搜"收入"时 `finance_expenses.rd_expense` 也有一定语义相关性）
2. 提高精度：候选集越小，Top-K 的质量越高
3. 与表召回解耦：表召回负责"选对表"，字段匹配只负责在表内"选对字段"

三、混合匹配：向量相似度 + 业务规则

纯向量匹配的局限

即使有丰富的字段描述，纯 Embedding 相似度仍然有局限：

- 无法强制执行："收入 = net_amount"是企业的财务制度，不是"语义相近"能保证的
- 无法绝对排除：`gross_amount` 的描述中出现了"销售额"（用于说明"不应使用"），反而可能被错误召回
- 缺乏条件逻辑："涉及多币种时必须引入汇率表"——这是条件规则，不是相似度问题

业务规则设计

引入三种规则类型：

类型

作用

示例

whitelist

强制包含

提到"收入/销售额" → 必须包含 net_amount

blacklist

强制排除

提到"收入/销售额" → 排除 gross_amount

conditional

条件触发

提到"人民币/换算" → 包含 exchange_rates 相关字段

规则定义格式：

```
BUSINESS_RULES = [
    {
        "type": "whitelist",
        "trigger_keywords": ["收入", "销售额", "营业收入", "营收"],
        "force_include": ["sales_orders.net_amount"],
        "reason": "收入口径统一使用 net_amount（不含税）",
    },
    {
        "type": "blacklist",
        "trigger_keywords": ["收入", "销售额", "营业收入", "营收"],
        "force_exclude": ["sales_orders.gross_amount"],
        "reason": "除非明确要求含税，否则排除 gross_amount",
    },
    # ...
]
```

混合评分公式

$$\text{final_score} = (1 - \alpha) \times \text{embedding_score} + \alpha \times \text{rule_score}$$

其中：

- embedding_score：向量相似度（0~1）
- rule_score：规则分数——强制包含 = 1.0、强制排除 = -1.0、无规则 = 0.0
- α (rule_weight)：规则权重，默认 0.3

设计意图：

- 当规则和向量一致时（如 `net_amount` 向量分高 + 白名单加分），结果更确定
- 当规则和向量冲突时（如 `gross_amount` 向量分不低但被黑名单），规则能将其压到阈值以下
- 白名单优先级高于黑名单：如果同一字段同时被 `include` 和 `exclude`，保留 `include`

评分示例

假设用户问"查询上个月各大区的销售额"：

字段

`embedding_score`

`rule_score`

`final_score` ($\alpha=0.3$)

`sales_orders.net_amount`

0.62

+1.0（白名单）

$$0.62 \times 0.7 + 1.0 \times 0.3 = 0.734$$

`sales_orders.gross_amount`

0.58

-1.0（黑名单）

$$0.58 \times 0.7 + (-1.0) \times 0.3 = 0.106$$

`sales_orders.region`

0.55

0（无规则）

$$0.55 \times 0.7 + 0 = 0.385$$

`sales_orders.order_date`

0.48

0

0.336

阈值设为 0.25 时，gross_amount 的 0.106 仍然通过——但排在最后；如果阈值设为 0.2 则勉强保留。实际效果中，0.106 的 final_score 已经远低于 net_amount 的 0.734，模型选择正确字段的概率大幅提升。

强制补充机制

如果某个被白名单指定的字段恰好不在向量检索的 Top-K 结果中（可能因为 K 太小或描述文本和问题差距大），系统会主动补充它：

```
# 补充强制包含但未被向量检索命中的字段
for field_key in force_include:
    if field_key not in seen_keys:
        meta = FIELD_METADATA.get(field_key)
        if meta and (not candidate_tables or meta["table"] in candidate_tables):
            scored_fields.append({
                "field_key": field_key,
                "score": rule_weight * 1.0, # 纯规则分
                "rule_applied": "强制包含（补充）",
                ...
            })
```

这保证了业务规定必须使用的字段永远不会被遗漏。

四、表+字段串联召回

集成接口

retrieve_schema() 是本课的核心对外接口，将表召回和字段匹配串联为一个完整的流程：

```
def retrieve_schema(query: str) -> dict:
    # 1. 表召回
    tables = retrieve_tables(query, top_k=3, score_threshold=0.3)
    candidate_table_names = [t["table_name"] for t in tables]

    # 2. 字段匹配（限定在召回的表范围内）
    fields = match_fields(query, candidate_tables=candidate_table_names, top_k=10)

    # 3. 组装精简 Schema 片段
    schema_snippet = _build_schema_snippet(tables, fields)

    return {"tables": tables, "fields": fields, "schema_snippet": schema_snippet}
```

输出格式

```
result = retrieve_schema("查询上个月各大区的销售额")

# result["schema_snippet"] 输出：
# 表: sales_orders（相关字段: net_amount, region, order_date, order_status）
# 表: dim_customers（相关字段: region）
```

对比全量 Schema（60 行、约 1500 tokens）→ 精简 Schema 片段（2 行、约 50 tokens）。Token 消耗降低了 96%，且包含了所有必要信息。

五、效果验证

经典歧义案例

完整代码文件路径：code/chatbi_mvp/field_matcher.py

```
uv run field_matcher.py
```

输出示例：

"查询各大区的销售额"：net_amount 被白名单强制包含（总分 0.540），gross_amount 被黑名单强制排除（未出现在结果中），region 在向量检索中也排名靠前

```
=====
问题：查询上个月各大区的销售额
关注点：应选 net_amount 而非 gross_amount，应选 region 而非 country
=====
召回表：['dim_customers', 'sales_orders', 'exchange_rates']
匹配字段（Top-5）：
sales_orders.net_amount          总分:0.540 向量:0.343 [强制包含]
sales_orders.order_status        总分:0.516 向量:0.308 [强制包含]
dim_customers.region             总分:0.339 向量:0.484
sales_orders.region              总分:0.292 向量:0.417
sales_orders.quantity            总分:0.258 向量:0.369

精简Schema：
表：dim_customers（相关字段：region, customer_type, industry）
表：sales_orders（相关字段：net_amount, order_status, region, quantity, order_date, order_no, discount_amount）
表：exchange_rates
```

"各产品线的毛利率"：material_cost 和 labor_cost 被成本规则强制包含（总分分别为 0.588 和 0.569），远高于纯向量分的字段

```
=====
问题：各产品线的毛利率
关注点：应包含 net_amount + material_cost + labor_cost + quantity
=====
召回表：['dim_products', 'finance_expenses', 'dim_customers']
匹配字段（Top-5）：
dim_products.material_cost       总分:0.588 向量:0.411 [强制包含]
dim_products.labor_cost          总分:0.569 向量:0.384 [强制包含]
dim_products.product_line        总分:0.327 向量:0.467
dim_products.standard_cost       总分:0.272 向量:0.389
dim_products.category            总分:0.265 向量:0.379

精简Schema：
表：dim_products（相关字段：material_cost, labor_cost, product_line, standard_cost, category, product_name, tech_route）
表：finance_expenses（相关字段：selling_expense, logistics_expense, marketing_expense）
```

表: dim_customers

"含税收入": net_amount 和 gross_amount
同时被白名单包含——因为"含税"触发了含税场景例外规则, gross_amount 被主动纳入而非排除

=====

问题: 按客户类型统计含税收入

关注点: 明确说含税, 应选 gross_amount

召回表: ['dim_customers', 'sales_orders', 'finance_expenses']

匹配字段 (Top-5):

sales_orders.net_amount	总分:0.623 向量:0.462 [强制包含]
sales_orders.gross_amount	总分:0.598 向量:0.425 [强制包含]
sales_orders.order_status	总分:0.553 向量:0.362 [强制包含]
dim_customers.customer_type	总分:0.367 向量:0.524
dim_customers.industry	总分:0.324 向量:0.462

精简Schema:

表: dim_customers (相关字段: customer_type, industry, customer_name, customer_id)

表: sales_orders (相关字段: net_amount, gross_amount, order_status, customer_id, quantity)

表: finance_expenses (相关字段: expense_id, ...)

业务规则评估演示

=====

业务规则评估演示

=====

问题: 查询各大区的销售额

强制包含: ['sales_orders.net_amount', 'sales_orders.order_status']

强制排除: ['sales_orders.gross_amount']

问题: 含税收入统计

强制包含: ['sales_orders.net_amount', 'sales_orders.order_status', 'sales_orders.gross_amount']

问题: 利润和成本分析

强制包含: ['dim_products.material_cost', 'dim_products.labor_cost']

对比全量 Schema 注入

维度

全量 Schema (第 5 课方式)

Schema Linking (表+字段)

注入内容

5 表 × 所有字段 = ~60 行

2-3 表 × 相关字段 = 2-3 行

Token 消耗

~1500 tokens

~50-100 tokens

歧义解决

依赖模型自行判断

规则强制约束 + 语义引导

模型负担

需从大量无关信息中提取

只看到相关信息，注意力集中

六、规则维护建议

规则从哪来

业务规则不是开发者凭空设想的，而是来自：

1. 财务制度文档：定义了"收入""成本""利润"的口径
2. 数据字典：标注了字段的取值范围和使用约定
3. 历史错误案例：每次 SQL 生成出错后，分析原因并沉淀为规则

规则数量控制

规则不是越多越好。过多规则会导致：

- 规则之间可能冲突
- 维护成本增加
- 触发关键词误命中

建议控制在 10-20 条核心规则，覆盖最常见的歧义场景即可。

小结

本节课完成了 Schema Linking 的第二步——字段匹配：

三个关键设计决策：

1. 字段描述文本质量决定匹配质量——投入时间在描述构造上，回报远大于调参数
2. 候选范围限定——只在召回的表内匹配字段，避免全局搜索的噪声
3. 混合评分——向量相似度提供"大概率正确"，业务规则提供"绝对约束"

下一课将实现 Schema Linking 的第三步：当查询涉及多张表时，如何自动推理 Join 路径和条件。

附件（飞书无下载权限）：第 15 课：从手写到框架——LangChain + ChromaDB 向量检索工程化.pdf

附件（飞书无下载权限）：第 16 课：字段语义匹配——解决_amount 到底是哪种金额_.pdf